



FastFood POS

Point-of-Sale & Billing System

Full Feature Documentation

Technology: Node.js · Express.js · SQLite3 · EJS · Tailwind CSS

Version: 1.0.0 | Generated: 09 June 2026



Role-Based



Thermal Receipt



Analytics

Table of Contents

Overview & Setup

1. Project Overview	3
2. Technology Stack	3
3. Installation & Configuration	4
4. Database Schema	5

Core Features

5. Authentication & Role-Based Access	6
6. Dashboard	7
7. POS Ordering System	8
8. Cart Management	9
9. Payment Processing	10
10. Bill Generation & Receipts	11

Admin Features

11. Menu Management	12
12. Category Management	13
13. Order History & Search	14
14. Daily Sales Reports	15

Technical Reference

15. API Endpoints Reference	16
16. Frontend Architecture (pos.js)	18
17. Project File Structure	19
18. Default Seed Data	20

1 Project Overview

FastFood POS & Billing System is a complete, production-ready point-of-sale and billing solution built for fast food restaurants and food outlets. It provides an intuitive touchscreen-friendly ordering interface, thermal-printer-ready receipts with UPI QR codes, comprehensive inventory/menu management, and real-time daily sales analytics — all through a secure, role-based web interface accessible from any device on the local network.



For Cashiers

Quick order entry with category filters and keyboard shortcuts. Real-time cart with GST calculation. One-click bill generation with printable receipt. Order history lookup.



For Admins

Full menu & category management. Activate/deactivate items instantly. View daily sales reports with payment breakdowns and top-selling items. Cancel orders.



Receipt & Payment

80mm thermal-printer-ready layout. UPI QR code auto-generated on receipt. Cash change calculator. Supports Cash, UPI, and Card payments.



Analytics

Today's revenue, order count, and averages on the dashboard. Daily report with hourly breakdown, top 10 items, and payment method split.

Key Highlights



Mobile-First Responsive Design

Works seamlessly on tablets, phones, and desktops. Bottom-sheet cart drawer on mobile, side cart on desktop.



Keyboard Shortcuts

Press **/** to focus search, **Enter** to add the top match to cart. Designed for high-speed order entry.



Secure & Role-Based

Bcrypt-hashed passwords, session-based auth, 8-hour expiry. Cashiers cannot access admin features.



Zero Setup Database

SQLite with WAL mode. Database and seed data created automatically on first run — no external DB server needed.

2 Technology Stack

Layer	Technology	Version	Purpose
Runtime	Node.js	v18+	Server-side JavaScript runtime
Framework	Express.js	~4.16.1	Web application framework, routing
Template Engine	EJS	~2.6.1	Server-side HTML rendering
Database	SQLite3	^6.0.1	Embedded relational database (WAL mode)
Styling	Tailwind CSS	CDN v3	Utility-first responsive CSS framework
Session Auth	express-session	^1.19.0	User session management (8-hour expiry)
Password Hash	bcryptjs	^3.0.3	Secure password hashing & comparison
Flash Messages	connect-flash	^0.1.1	One-time notification messages
QR Code	qrcode	^1.5.4	UPI QR code generation on receipts
HTTP Override	method-override	^3.0.0	PUT/DELETE via HTML forms
Logging	morgan	~1.9.1	HTTP request logging
Env Config	dotenv	^17.4.2	Environment variable management

Architecture Pattern

The app follows the **MVC (Model-View-Controller)** pattern:

Backend (Server-Side)

- **Models** — SQLite tables via `database/db.js`
- **Controllers** — Express route handlers in `routes/`
- **Views** — EJS templates in `views/`
- **Middleware** — Auth guards in `middleware/auth.js`

Frontend (Client-Side)

- **Tailwind CSS** — responsive layouts & components
- **Vanilla JS** — cart state, totals, API calls
- **pos.js** — full client-side POS logic
- **EJS partials** — reusable UI components

No Build Step Required

This project uses Tailwind CSS via CDN and vanilla JavaScript — no webpack, no build process. Just `npm install` and `npm start`.

3 Installation & Configuration

Quick Start

```
# Clone / navigate to project cd fast-food # Install dependencies npm install # Start the server (auto-creates database & seed data) npm start # Open in browser http://localhost:4000
```

NPM Scripts

Script	Command	Description
<code>npm start</code>	<code>node ./bin/www</code>	Start server on port 4000
<code>npm run kill</code>	<code>netstat / taskkill</code>	Free port 4000 if occupied (Windows)
<code>npm run fresh</code>	<code>kill + start</code>	Kill existing process then start fresh

Environment Variables (.env)

```
UPI_ID=yourname@bank # UPI address printed on receipts SHOP_NAME=Fast Food Shop # Shop name shown on receipts & QR PORT=4000 # HTTP server port SESSION_SECRET=your-secret # Session encryption key (change in production!)
```

Default Login Credentials

Role	Username	Password	Access Level
ADMIN	admin	admin123	Full access: menu, categories, reports, order cancellation
CASHIER	cashier	cashier123	New orders, order history, receipt viewing

⚠ Security Note: Change the default passwords and SESSION_SECRET before deploying on a network accessible to customers or untrusted users. The default credentials are only for initial setup.

System Requirements

- Node.js v18 or higher
- Windows / macOS / Linux
- ~50 MB disk space (SQLite database grows with orders)
- Modern browser (Chrome, Firefox, Edge, Safari)
- Network access for staff to connect from POS terminals

4 Database Schema

The application uses **SQLite3** with **WAL (Write-Ahead Logging)** mode for better read concurrency. The database is auto-created at `database/fastfood.db` on first run.

Table: users

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique user identifier
username	TEXT	UNIQUE NOT NULL	Login username
password_hash	TEXT	NOT NULL	Bcrypt-hashed password
role	TEXT	DEFAULT 'cashier'	Values: admin cashier
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Table: categories

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Category identifier
name	TEXT	UNIQUE NOT NULL	Category display name (e.g. "Burgers")
emoji	TEXT	DEFAULT '🍔'	Emoji icon shown on POS screen
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Category creation time

Table: menu_items

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Item identifier
category_id	INTEGER	NOT NULL, FOREIGN KEY	References categories.id
name	TEXT	NOT NULL	Item name
price	REAL	NOT NULL	Unit price in rupees
gst	REAL	DEFAULT 5	GST percentage (e.g. 5 = 5%)
status	TEXT	DEFAULT 'active'	Values: active inactive

Table: orders

Column	Type	Description
id	INTEGER PK	Auto-incremented order ID
order_number	TEXT UNIQUE	Format: ORD + YYYYMMDDHHMMSS + 2 random digits
customer_name	TEXT	Optional customer name
mobile	TEXT	Optional mobile number
order_type	TEXT	Dine In Takeaway Delivery
table_number	TEXT	Optional table number (Dine In only)
subtotal	REAL	Total before GST
gst	REAL	Total GST amount collected
discount	REAL	Discount applied (₹)
total	REAL	Final payable amount
payment_method	TEXT	Cash UPI Card
cash_received	REAL	Cash tendered (cash payments only)
status	TEXT	completed cancelled
created_at	DATETIME	Order timestamp

Table: order_items

Column	Type	Description
id	INTEGER PK	Auto-incremented
order_id	INTEGER FK	References orders.id
item_id	INTEGER	Original menu item ID (snapshot)
item_name	TEXT	Name at time of order (immutable)
quantity	INTEGER	Number of units ordered
price	REAL	Unit price at time of order
gst	REAL	GST % at time of order
total	REAL	quantity × price (subtotal)

5 Authentication & Role-Based Access Control

Authentication Flow

1. User visits any route → middleware checks session for `req.session.user`
2. If not logged in → redirected to `/login`
3. On login form submit → username & bcrypt password compared against `users` table
4. On success → user object stored in session, redirect to `/dashboard`
5. On failure → flash error message shown on login page
6. Session expires after **8 hours** of inactivity (configurable via `maxAge`)

Middleware Guards

Middleware	File	Applied To	Behavior
<code>requireAuth</code>	<code>middleware/auth.js</code>	All routes except <code>/login</code>	Redirects unauthenticated users to <code>/login</code>
<code>requireAdmin</code>	<code>middleware/auth.js</code>	Admin-only routes	Returns 403 Forbidden for non-admin users

Role-Based Feature Access

Feature	ADMIN	CASHIER
Login / Logout	✓	✓
Dashboard	✓	✓
New Order (POS)	✓	✓
View Order History	✓	✓
View Receipt	✓	✓
Cancel Order	✓	✗
Menu Item Management	✓	✗
Category Management	✓	✗
Daily Sales Report	✓	✗

Security Features



Password Security



Session Security

All passwords hashed with **bcryptjs**. Plain-text passwords never stored. bcrypt's built-in salt prevents rainbow table attacks.

Sessions signed with a `SESSION_SECRET` key. Configurable expiry. Sessions stored server-side (memory store).

Route Protection

Every route applies `requireAuth`. Admin routes additionally apply `requireAdmin`. 403 returned on unauthorized access.

Flash Messages

One-time session messages for login errors, success confirmations, and validation feedback. Auto-cleared after display.

6 Dashboard

The dashboard at `/dashboard` is the home screen after login. It provides a real-time snapshot of today's business activity.

Today's Stats Cards



Total Orders Today



Revenue Today (INR)



Active Menu Items

Dashboard Components



Statistics Cards

Four cards showing: Total orders today, Today's revenue (₹ Indian locale format), Active menu items count, Total categories count. Revenue excludes cancelled orders.



Quick Actions Panel

Prominent "New Order" button for cashiers. Links to "View All Orders" and "Daily Report" (admin only). One-click navigation to most-used features.



Recent Orders Table

Shows the 10 most recent orders with: Order number (clickable → receipt), customer name, payment method badge, total amount, and relative time. Empty state shown if no orders today.



Responsive Navigation Sidebar

Desktop: full sidebar with icons and labels. Mobile: top bar with menu icon. Nav items show only accessible routes based on user role.

Dashboard SQL Queries

Stat	Query Logic
Today's Orders	COUNT of orders WHERE status='completed' AND date(created_at) = today
Today's Revenue	SUM(total) WHERE status='completed' AND date(created_at) = today
Active Items	COUNT of menu_items WHERE status='active'
Categories	COUNT of all categories
Recent Orders	SELECT last 10 completed orders with JOIN on items

7 POS Ordering System

The POS screen at `/orders/new` is the core feature of the application. It is optimized for fast order entry on both desktop and mobile/tablet devices.

Menu Browsing



Category Filter Tabs

Horizontal scrollable row of category buttons with emoji icons. "All" tab shows every active item. Clicking a category instantly filters the grid. Active category is highlighted in orange.



Real-Time Search

Search box filters menu items by name as you type. Works across all categories simultaneously. Press `/` anywhere to focus the search box. Press **Escape** to clear search. Press **Enter** to add the first matching item to cart.



Menu Item Cards

Responsive grid of item cards showing name and price. Click any card to add to cart. Card flashes orange to confirm the add. Only active items are shown. Items in cart show a count badge.

Customer Information (Optional)

Field	Type	Description
Customer Name	Text input	Optional — shown on receipt if entered
Mobile Number	Number input	Optional — printed on receipt for reference
Order Type	Dropdown	Dine In / Takeaway / Delivery
Table Number	Text input	Visible only when Order Type = "Dine In"

Responsive Layout

Desktop / Tablet

- Two-column layout
- Menu grid on the left (main area)
- Cart panel fixed on the right
- Cart always visible while browsing
- Wide category filter bar at top

Mobile

- Full-width menu grid
- Cart icon in top bar shows item count badge
- Tap cart icon → bottom sheet slides up
- Drag handle on sheet for touch gesture
- Cart overlays menu (dismissable)

 **Speed Tip:** Cashiers can enter a full order using only the keyboard: press **/** to focus search, type item name, press **Enter** to add, repeat. No mouse required for common items.

8 Cart Management

Cart Features



Add Items

Click any menu card to add. If item already in cart, quantity increments by 1. Visual flash confirms the add. Cart total updates instantly.



Adjust Quantities

Each cart item has – and + buttons. Click – to decrement (removes item at 0). Click + to increment. Quick quantity presets available for bulk orders (e.g. 100, 500 units for high-volume items).



Remove Items

Each cart row has a remove (x) button for direct deletion. Quantity going below 1 via the – button also removes the item.



Clear Cart

Clear all button empties the entire cart. Confirmation dialog prevents accidental clears.

Price Calculation

Field	Formula	Notes
Item Subtotal	quantity × unit price	Shown per line in cart
Subtotal	Sum of all item subtotals	Before tax
GST	Each item: (qty × price × gst%) / 100, summed	Per-item GST rate configured in menu
Discount	Manual entry in ₹	Deducted from total
Total	Subtotal + GST – Discount	Final payable amount
Change	Cash Received – Total	Only shown for Cash payments

Discount & Cash Change

Discount

- Enter discount amount in ₹ directly
- Total recalculates in real-time
- Discount is stored with the order record
- Visible on printed receipt

Cash Change Calculator

- Visible only when Cash payment selected
- Enter cash received by customer
- Change due shown instantly
- Cash received stored with order

9 Payment Processing

The system supports three payment methods, selectable via toggle buttons in the cart/billing panel before generating the bill.

Payment Methods

Cash

- Most common method
- Enter cash received by customer
- Change due calculated & shown instantly
- Cash received stored in `orders.cash_received`
- Change printed on receipt

UPI

- Generates QR code on receipt
- QR encodes full UPI deep-link
- Customer scans with any UPI app
- UPI ID configured in `.env`
- No cash entry needed

Card

- Simple flag — no POS terminal integration
- Records payment as "Card"
- No additional fields required
- Card label shown on receipt

UPI QR Code Generation

When payment method is UPI, the receipt displays a scannable QR code. The QR code is generated using the **qrcode** npm package and encodes:

```
# UPI Payment URL format upi://pay?pa={UPI_ID}&pn={SHOP_NAME}&am={ORDER_TOTAL}&cu=INR #  
Example upi://pay?pa=yourshop@bank&pn=Fast+Food+Shop&am=350.00&cu=INR
```

The QR code is generated server-side as a Base64 Data URL and embedded directly in the receipt HTML — no external service is called. Works offline.

Payment Method in Reporting

All three payment methods are tracked in the database and included in the daily sales report's payment breakdown, showing count and total amount per method with percentage bars.

10 Bill Generation & Receipts

Bill Generation Process

1. Cashier clicks "**Generate Bill**" button in cart panel
2. Client-side JS posts cart JSON to `POST /orders`
3. Server validates, stores order + items in database
4. Unique order number generated: ORD + YYYYMMDDHHMMSS + 2 random digits
5. If UPI payment: QR code Data URL generated with **qrcode** library
6. Browser redirected to `/orders/{id}/receipt`
7. Receipt rendered in browser — user clicks **Print**

Receipt Contents

Section	Content
Header	Shop name (from SHOP_NAME env), "TAX INVOICE" label
Order Info	Order number, date & time
Customer Info	Name, mobile (if provided)
Order Type	Dine In / Takeaway / Delivery, table number if applicable
Item List	Item name, qty × unit price = line total
Pricing	Subtotal, GST amount, discount, Grand Total
Payment	Payment method, cash received, change due
UPI QR	Scannable QR code (only for UPI orders)
Footer	"Thank you for visiting!" message

Thermal Printer Support

The receipt is designed for **80mm thermal printers**. It uses a narrow-width layout, monospace-friendly fonts, and print-optimized CSS. When printed from Chrome/Edge, it produces a clean thermal receipt with no headers/footers/margins.

- Max width: 80mm (print media query override)
- Sidebar, navigation, buttons hidden on print
- High contrast black & white output
- Dotted separator lines between sections
- Centered alignment for header and totals

Unique Order Numbers

Format: ORD + YYYYMMDDHHMMSSxx (18 characters) **ORD2024061514305247** # YYYYMMDDHHMMSS = date/time of creation # xx = random 2-digit suffix (10-99) for collision avoidance

11 Menu Management (Admin Only)

Accessible at `/menu`. Allows admins to manage the full menu catalog — adding, editing, toggling, and removing items.

Add / Edit Menu Item

Field	Type	Required	Description
Name	Text	Yes	Item display name shown on POS & receipts
Category	Dropdown	Yes	Assigns item to a category for filtering
Price	Number (₹)	Yes	Unit selling price in Indian Rupees
GST %	Number (%)	No	Per-item tax rate, default 5%
Status	Toggle	Auto	Active (visible on POS) or Inactive (hidden)

Menu Item Features



Search & Filter

Search by name, filter by category, filter by status (active/inactive). Helps manage large menus quickly.



Quick Status Toggle

Toggle switch on each item row calls `POST /menu/:id/toggle` via AJAX. No page reload — instant availability control. Useful for "sold out" scenarios.



Edit in Place

Edit button opens a form pre-filled with item data. Submit updates via `POST /menu/:id/edit`. Updated item appears immediately.



Delete Item

Delete button removes item via `DELETE /menu/:id`. Item removed from POS menu immediately. Historical order data is preserved (order_items stores name snapshot).

⚠ Note on Deletion: Deleting a menu item does not affect past orders — the item name and price are stored as a snapshot in `order_items.item_name` and `order_items.price` at order time.

12 Category Management (Admin Only)

Accessible at `/categories` . Organize menu items into browsable groups shown as filter tabs on the POS screen.

Category Features



Add Category

Enter a category name and select an emoji from the picker grid. The emoji appears as an icon next to the category name on the POS filter tabs.



Emoji Picker

A grid of preset food-related emojis (🍔 🍕 🥪 🌮 🍹 🍷 🍺 🍻 🍽️ etc.) with a click-to-select interface. Custom emoji can also be typed directly. Makes POS navigation visually intuitive.



Edit Category

Update category name and/or emoji. Changes reflected immediately on the POS screen category filter tabs.



Delete Category (Safe)

A category can only be deleted if it has **zero menu items** linked to it. Prevents orphaned items. Shows an error if items exist in the category.



Item Count Display

Each category card shows the count of menu items assigned to it. Helps identify empty or large categories at a glance.

Default Categories (Seed Data)

Category	Emoji	Sample Items
Burgers	🍔	Veg, Chicken, Paneer, Double Patty
Pizza	🍕	Margherita, BBQ Chicken, Veggie Supreme
Sandwiches	🥪	Veg, Chicken Club, Grilled
Rolls	🌮	Egg Roll, Chicken Roll, Paneer Roll
Drinks	🍹	Coke, Pepsi, Sprite, Lassi, Water
Fries	🍷	Small, Medium, Large Fries
Combo Meals	🍺	Burger+Fries+Drink, Combo A/B/C

13 Order History & Search

Accessible at `/orders` . Lists recent orders with powerful filtering to find any past order.

Search & Filter Options

Filter	Type	Description
Search Query	Text input	Searches order number, customer name, and mobile number. Case-insensitive partial match.
Date From	Date picker	Show orders on or after this date
Date To	Date picker	Show orders on or before this date
Payment Method	Dropdown	Filter by Cash / UPI / Card (or All)

Order History Table Columns

Column	Description
Order #	Clickable link → opens receipt page
Date & Time	Order creation timestamp (formatted)
Customer	Customer name or "—" if not provided
Type	Dine In / Takeaway / Delivery badge
Payment	Cash / UPI / Card badge with color coding
Total	Final order amount in ₹
Status	Completed (green) / Cancelled (red) badge
Actions	View receipt button; Cancel button (admin only)

Order Cancellation

Admin Only Feature. Any completed order can be cancelled via `POST /orders/:id/cancel` . The order status is set to cancelled — it is **not deleted**. Cancelled orders are excluded from revenue calculations but remain visible in history for audit purposes.

Pagination & Limits

The history view returns up to **100 most recent orders** by default (descending by created_at). Filters reduce this set further. For large deployments, this ensures page load performance remains fast.

14 Daily Sales Reports (Admin Only)

Accessible at </reports/daily>. Select any date to view a comprehensive breakdown of that day's sales.

Report Sections

Summary Statistics

Metric	Description
Total Orders	Count of completed orders for the selected date
Total Revenue	Sum of all completed order totals (incl. GST)
GST Collected	Sum of all GST amounts for the day
Average Order Value	Total Revenue ÷ Total Orders

Payment Method Breakdown

- Count and total amount per payment method (Cash / UPI / Card)
- Percentage of total revenue per method
- Visual progress bar showing relative share
- Helps understand customer payment preferences

Top 10 Selling Items

- Ranked by quantity sold (descending)
- Shows: rank, item name, total quantity sold, total revenue from item
- Aggregated from all completed orders on that date
- Useful for inventory planning and identifying bestsellers

Hourly Sales Chart

- Breaks down orders and revenue by hour of day (00:00–23:00)
- Shows peak and off-peak hours visually
- Helps with staffing decisions and prep scheduling
- Only hours with at least one order are shown

Date Selection

A date picker at the top of the report defaults to today's date. Select any past date to view historical data. The URL updates with the selected date for bookmarking/sharing.

 **Analytics are real-time:** The report queries the live database — there is no pre-computed cache. Revenue figures are always accurate to the current moment.

15 API Endpoints Reference

Authentication Routes

Method	Path	Description	Access
GET	/login	Render login page	Public
POST	/login	Authenticate user, create session	Public
GET	/logout	Destroy session, redirect to login	Auth

Dashboard Routes

Method	Path	Description	Access
GET	/dashboard	Dashboard with today's stats & recent orders	Auth

Category Routes

Method	Path	Description	Access
GET	/categories	List all categories with item counts	Admin
POST	/categories	Create new category	Admin
POST	/categories/:id/edit	Update category name & emoji	Admin
DELETE	/categories/:id	Delete category (fails if items exist)	Admin

Menu Routes

Method	Path	Description	Access
GET	/menu	List menu items with search/filter	Admin
POST	/menu	Create new menu item	Admin
POST	/menu/:id/edit	Update menu item details	Admin
DELETE	/menu/:id	Delete menu item	Admin
POST	/menu/:id/toggle	Toggle active/inactive status (JSON API)	Admin

Order Routes

Method	Path	Description	Access
GET	/orders/new	Render POS ordering screen	Auth
POST	/orders	Create order, returns JSON {orderId}	Auth
GET	/orders	Order history with search/filter	Auth
GET	/orders/:id/receipt	View printable receipt (UPI QR if applicable)	Auth
POST	/orders/:id/cancel	Cancel order (status → cancelled)	Admin

Report Routes

Method	Path	Description	Access
GET	/reports/daily	Daily report for selected date	Admin

POST /orders — Request Body

```
{ "customer_name": "Raj Kumar", // optional "mobile": "9876543210", // optional
  "order_type": "Dine In", // "Dine In" | "Takeaway" | "Delivery" "table_number": "T5", //
  optional "payment_method": "Cash", // "Cash" | "UPI" | "Card" "cash_received": 500, // only
  for Cash "discount": 20, // optional, default 0 "items": [ { "id": 3, "name": "Veg Burger",
  "price": 80, "gst": 5, "qty": 2 }, { "id": 15, "name": "Coke", "price": 40, "gst": 12,
  "qty": 2 } ] }
```

16 Frontend Architecture (pos.js)

The POS screen's client-side logic lives in `public/js/pos.js`. It manages all cart state, UI interactions, and the order submission flow entirely in-browser.

State Variables

Variable	Type	Description
cart	Array	Array of cart items: {id, name, price, gst, qty}
paymentMethod	String	Selected payment: 'Cash' 'UPI' 'Card'
activeCat	Number/null	Currently selected category ID (null = All)
cartOpen	Boolean	Mobile cart drawer visibility state

Key Functions

Function	Description
addToCart(item)	Add item or increment quantity. Triggers card flash animation.
changeQty(itemId, delta)	Adjust quantity by delta. Removes item if qty reaches 0.
removeItem(itemId)	Remove specific item from cart entirely.
clearCart()	Prompt confirmation then empty entire cart.
filterCategory(catId)	Set activeCat, call applyFilters() to re-render grid.
applyFilters()	Apply both category and search text filters to menu grid.
setupSearch()	Bind input event for live search. Enter key adds top result.
setupKeyboard()	Global '/' key → focus search. Escape → clear search.
renderCart()	Re-render cart item list with qty controls and line totals.
updateTotals()	Calculate and display subtotal, GST, discount, total, change.
setPayment(method)	Update paymentMethod, toggle button styles, show/hide cash field.
generateBill()	POST cart JSON to /orders API, redirect to receipt on success.
openCart()	Show mobile bottom-sheet cart drawer with slide animation.
closeCart()	Hide mobile cart drawer.

Event Flow: Adding an Item

1. User clicks a menu card (or presses Enter in search)
2. `addToCart(item)` called with item data
3. Cart array updated (push new item or increment qty)
4. `renderCart()` re-renders cart list HTML
5. `updateTotals()` recalculates all price fields
6. Card element flashes orange for 300ms (visual feedback)
7. Mobile: cart badge count updates

Event Flow: Generating Bill

1. Cashier clicks "Generate Bill"
2. Validates cart is non-empty
3. `generateBill()` collects form data + cart
4. Sends POST `/orders` with JSON body
5. On success (200): redirects to `/orders/{id}/receipt`
6. On error: shows alert with error message

17 Project File Structure

```
fast-food/ ├── bin/ | └─ www # HTTP server entry point (port 4000) ├── database/ | ├──
db.js # SQLite init, schema, seed data | └─ fastfood.db # Auto-created SQLite database file
├── middleware/ | └─ auth.js # requireAuth, requireAdmin guards ├── public/ | ├── js/ | |
└─ pos.js # POS client-side logic (cart, filters, API) | └─ stylesheets/ | └─ style.css #
Custom CSS overrides ├── routes/ | ├── auth.js # GET/POST /login, GET /logout | ├──
dashboard.js # GET /dashboard | ├── categories.js # CRUD for categories | ├── menu.js # CRUD
for menu items, toggle API | ├── orders.js # POS new order, create, history, receipt, cancel
| └─ reports.js # GET /reports/daily ├── views/ | ├── auth/login.ejs # Login form | ├──
categories/index.ejs # Category management UI | ├── dashboard/index.ejs # Home with stats &
recent orders | ├── menu/index.ejs # Menu item management UI | ├── orders/ | | ├── pos.ejs #
Main POS ordering screen | | ├── history.ejs # Order history & search | | └─ receipt.ejs #
Printable receipt (80mm thermal) | ├── reports/daily.ejs # Daily analytics report | └─
partials/ | ├── head.ejs # HTML head, Tailwind CDN | ├── sidebar.ejs # Navigation sidebar |
└─ footer.ejs # Page footer | └─ _emoji_grid.ejs # Emoji picker component ├── .env #
UPI_ID, SHOP_NAME, PORT, SESSION_SECRET ├── app.js # Express app setup, middleware, routes
└─ package.json # Dependencies & npm scripts
```

Key Architectural Decisions



WAL Mode for SQLite

Write-Ahead Logging enabled for better read/write concurrency — prevents database locks during simultaneous order creation and report queries.



Order Item Snapshots

Order items store name and price at time of order, not just a foreign key. This preserves historical accuracy even if menu items are later edited or deleted.



Method Override

HTML forms only support GET/POST. The `method-override` middleware allows DELETE requests via a hidden `_method` field, enabling RESTful routes without JavaScript fetch.

18 Default Seed Data

On first run, `database/db.js` automatically seeds the database with 2 users, 7 categories, and 23 menu items if the tables are empty.

Menu Items by Category

Burgers

Item	Price	GST
Veg Burger	₹80	5%
Chicken Burger	₹120	5%
Paneer Burger	₹100	5%
Double Patty Burger	₹150	5%

Pizza

Item	Price
Margherita	₹150
BBQ Chicken	₹220
Veggie Supreme	₹180

Sandwiches

Item	Price
Veg Sandwich	₹60
Chicken Sandwich	₹90
Club Sandwich	₹110

Rolls

Item	Price
Egg Roll	₹80
Chicken Roll	₹100

Drinks

Item	Price
Coke	₹40
Pepsi	₹40
Sprite	₹40
Lassi	₹60
Water Bottle	₹20

Fries

Item	Price
Small Fries	₹60
Medium Fries	₹80
Large Fries	₹100

Combo Meals

Item	Price	Item	Price
Paneer Roll	₹90	Combo A (Burger+Fries+Drink)	₹180
		Combo B (Chicken+Fries+Drink)	₹200
		Veg Combo	₹130

💡 **Customization:** All seed data can be replaced via the Admin panel without touching code. Add your own categories with custom emojis, set your real prices, and deactivate any items not on your menu.

FastFood POS & Billing System

Built with Node.js · Express · SQLite · Tailwind CSS

🔗 Designed for speed, simplicity, and reliability 🔗

FastFood POS & Billing System — Full Feature Documentation · End of Document